2
Data Structures

Data structures are the heart of any sophisticated program. Selecting the right data structure can make an enormous difference in the complexity of the resulting implementation. Pick the right data representation, and your task will be easy to program. Pick the wrong representation, and you may spend enormous amounts of time and code covering up for your lousy initial decision.

In this chapter, we will review the fundamental data structures which every programmer should be familiar with. We will motivate this discussion with an example program based on a children's card game. Many classic programming problems are based on games. After all, who makes it through their introductory programming courses without moving the towers of Hanoi, taking the knight's tour, or solving the eight queens problem?

2.1 Elementary Data Structures

Here we present the abstract operations of the most important data structures: stacks, queues, dictionaries, priority queues, and sets. We also describe the simplest way to implement these structures from scratch.

Be aware, however, that modern object-oriented languages like C++ and Java come with standard libraries of fundamental data structures. These will be briefly described in Section 2.2. It is worth every programmer's time to get acquainted with your libraries instead of repeatedly reinventing the wheel. Once you do, you can read this section with an eye for what each data structure is good for instead of the details of how it should be implemented.

2.1.1 Stacks

Stacks and queues are containers where items are retrieved according to the order of insertion, independent of content. *Stacks* maintain *last-in, first-out* order. The abstract operations on a stack include –

- *Push(x,s)* — Insert item x at the top of stack s.
- *Pop(s)* — Return (and remove) the top item of stack s.
- *Initialize(s)* — Create an empty stack.
- *Full(s), Empty(s)* — Test whether the stack can accept more pushes or pops, respectively.

Note that there is no element search operation defined on standard stacks and queues.

Defining these abstract operations enables us build a stack module to use and reuse without knowing the details of the implementation. The easiest implementation uses an array with an index variable to represent the top of the stack. An alternative implementation, using linked lists, is better because it can't overflow.

Stacks naturally model piles of objects, such as dinner plates. After a new plate is washed, it is pushed on the top of the stack. When someone is hungry, a clean plate is popped off the top. A stack is an appropriate data structure for this task since the plates don't care which one is used next. Thus one important application of stacks is whenever order *doesn't* matter, because stacks are particularly simple containers to implement.

Stack order is important in processing any properly nested structure. This includes parenthesized formulas (push on a "(", pop on ")"), recursive program calls (push on a procedure entry, pop on a procedure exit), and depth-first traversals of graphs (push on discovering a vertex, pop on leaving it for the last time).

2.1.2 Queues

Queues maintain *first-in, first-out* order. This appears fairer than last-in, first-out, which is why lines at stores are organized as queues instead of stacks. Decks of playing cards can be modeled by queues, since we deal the cards off the top of the deck and add them back in at the bottom. FIFO queues will be used in implementing breadth-first search in graphs in Chapter 9.

The abstract operations on a queue include —

- *Enqueue(x,q)* — Insert item x at the back of queue q.
- *Dequeue(q)* — Return (and remove) the front item from queue q
- *Initialize(q), Full(q), Empty(q)* — Analogous to these operation on stacks.

Queues are more difficult to implement than stacks, because action happens at both ends. The *simplest* implementation uses an array, inserting new elements at one end and *moving* all remaining elements to fill the empty space created on each dequeue.

However, it is very wasteful to move all the elements on each dequeue. How can we do better? We can maintain indices to the first (head) and last (tail) elements in the

array/queue and do all operations locally. There is no reason why we must explicitly clear previously used cells, although we leave a trail of garbage behind the previously dequeued items.

Circular queues let us reuse this empty space. Note that the pointer to the front of the list is always *behind* the back pointer! When the queue is full, the two indices will point to neighboring or identical elements. There are several possible ways to adjust the indices for circular queues. *All are tricky!* The easiest solution distinguishes full from empty by maintaining a count of how many elements exist in the queue:

```
typedef struct {
        int q[QUEUESIZE+1];             /* body of queue */
        int first;                      /* position of first element */
        int last;                       /* position of last element */
        int count;                      /* number of queue elements */
} queue;

init_queue(queue *q)
{
        q->first = 0;
        q->last = QUEUESIZE-1;
        q->count = 0;
}

enqueue(queue *q, int x)
{
        if (q->count >= QUEUESIZE)
                printf("Warning: queue overflow enqueue x=%d\n",x);
        else {
                q->last = (q->last+1) % QUEUESIZE;
                q->q[ q->last ] = x;
                q->count = q->count + 1;
        }
}

int dequeue(queue *q)
{
        int x;

        if (q->count <= 0) printf("Warning: empty queue dequeue.\n");
        else {
                x = q->q[ q->first ];
                q->first = (q->first+1) % QUEUESIZE;
                q->count = q->count - 1;
        }
```

```
        return(x);
}

int empty(queue *q)
{
        if (q->count <= 0) return (TRUE);
        else return (FALSE);
}
```

Queues are one of the few data structures which are easier to program using linked lists than arrays, because they eliminate the need to test for the wrap-around condition.

2.1.3 Dictionaries

Dictionaries permit content-based retrieval, unlike the position-based retrieval of stacks and queues. They support three primary operations –

- *Insert(x,d)* — Insert item x into dictionary d.
- *Delete(x,d)* — Remove item x (or the item pointed to by x) from dictionary d.
- *Search(k,d)* — Return an item with key k if one exists in dictionary d.

A data structures course may well present a dozen different ways to implement dictionaries, including sorted/unsorted linked lists, sorted/unsorted arrays, and a forest full of random, splay, AVL, and red-black trees – not to mention all the variations on hashing.

The primary issue in algorithm analysis is performance, namely, achieving the best possible trade-off between the costs of these three operations. But what we usually want in practice is the simplest way to get the job done under the given time constraints. The right answer depends on how much the contents of your dictionary change over the course of execution:

- *Static Dictionaries* — These structures get built once and never change. Thus they need to support search, but not insertion on deletion.

  The right answer for static dictionaries is typically an array. The only real question is whether to keep it sorted, in order to use binary search to provide fast membership queries. Unless you have tight time constraints, it probably isn't worth using binary search until $n > 100$ or so. You might even get away with sequential search to $n = 1,000$ or more, provided you will not be doing too many searches.

  Sorting algorithms and binary search always prove harder to debug than they should. Library sort/search routines are available for C, C++, and Java, and will be presented in Chapter 4.

- *Semi-dynamic Dictionaries* — These structures support insertion and search queries, but not deletion. If we know an upper bound on the number of elements to be inserted we can use an array, but otherwise we must use a linked structure.

Hash tables are excellent dictionary data structures, particularly if deletion need not be supported. The idea is to apply a function to the search key so we can determine *where* the item will appear in an array without looking at the other items. To make the table of reasonable size, we must allow for *collisions*, two distinct keys mapped to the same location.

The two components to hashing are (1) defining a hash function to map keys to integers in a certain range, and (2) setting up an array as big as this range, so that the hash function value can specify an index.

The basic hash function converts the key to an integer, and takes the value of this integer mod the size of the hash table. Selecting a table size to be a prime number (or at least avoiding obvious composite numbers like 1,000) is helpful to avoid trouble. Strings can be converted to integers by using the letters to form a base "alphabet-size" number system. To convert "steve" to a number, observe that e is the 5th letter of the alphabet, s is the 19th letter, t is the 20th letter, and v is the 22nd letter. Thus "steve" $\Rightarrow 26^4 \times 19 + 26^3 \times 20 + 26^2 \times 5 + 26^1 \times 22 + 26^0 \times 5 = 9,038,021$. The first, last, or middle ten characters or so will likely suffice for a good index. Tricks on how to do the modular arithmetic efficiently will be discussed in Chapter 7.

The absence of deletion makes *open addressing* a nice, easy way to resolve collisions. In open addressing, we use a simple rule to decide where to put a new item when the desired space is already occupied. Suppose we always put it in the next unoccupied cell. On searching for a given item, we go to the intended location and search sequentially. If we find an empty cell before we find the item, it does not exist anywhere in the table.

Deletion in an open addressing scheme is very ugly, since removing one element can break a chain of insertions, making some elements inaccessible. The key to efficiency is using a large-enough table that contains many holes. Don't be cheap in selecting your table size or else you will pay the price later.

- *Fully Dynamic Dictionaries* — Hash tables are great for fully dynamic dictionaries as well, provided we use *chaining* as the collision resolution mechanism. Here we associate a linked list with each table location, so insertion, deletion, and query reduce to the same problem in linked lists. If the hash function does a good job the m keys will be distributed uniformly in a table of size n, so each list will be short enough to search quickly.

2.1.4 Priority Queues

Priority queues are data structures on sets of items supporting three operations –

- *Insert(x,p)* — Insert item x into priority queue p.
- *Maximum(p)* — Return the item with the largest key in priority queue p.
- *ExtractMax(p)* — Return and remove the item with the largest key in p.

Priority queues are used to maintain schedules and calendars. They govern who goes next in simulations of airports, parking lots, and the like, whenever we need to schedule events according to a clock. In a human life simulation, it may be most convenient to determine when someone will die immediately after they are born. We can then stick this date in a priority queue so as to be reminded when to bury them.

Priority queues are used to schedule events in the sweep-line algorithms common to computational geometry. Typically, we use the priority queue to store the points we have not yet encountered, ordered by x-coordinate, and push the line forward one step at a time.

The most famous implementation of priority queues is the binary heap, which can be efficiently maintained in either a top-down or bottom-up manner. Heaps are very slick and efficient, but may be tricky to get right under time pressure. Far simpler is maintaining a sorted array, particularly if you will not be performing too many insertions.

2.1.5 Sets

Sets (or more strictly speaking *subsets*) are unordered collections of elements drawn from a given universal set U. Set data structures get distinguished from dictionaries because there is at least an implicit need to encode which elements from U are *not* in the given subset.

The basic operations on subsets are —

- *Member(x,S)* — Is an item x an element of subset S?
- *Union(A,B)* — Construct subset $A \cup B$ of all elements in subset A or in subset B.
- *Intersection(A,B)* — Construct subset $A \cap B$ of all elements in subset A and in subset B.
- *Insert(x,S), Delete(x,S)* — Insert/delete element x into/from subset S.

For sets of a large or unbounded universe, the obvious solution is representing a subset using a dictionary. Using sorted dictionaries makes union and intersection operations much easier, basically reducing the problem to merging two sorted dictionaries. An element is in the union if it appears at least once in the merged list, and in the intersection if it appears exactly twice.

For sets drawn from small, unchanging universes, bit vectors provide a convenient representation. An n-bit vector or array can represent any subset S from an n-element universe. Bit i will be 1 iff $i \in S$. Element insertion and deletion operations simply flip the appropriate bit. Intersection and union are done by "and-ing" or "or-ing" the corresponding bits together. Since only one bit is used per element, bit vectors can be space efficient for surprisingly large values of $|U|$. For example, an array of 1,000 standard four-byte integers can represent any subset on 32,000 elements.

2.2 Object Libraries

Users of modern object-oriented languages such as C++ and Java have implementations of these basic data structures available using standard library classes.

2.2.1 The C++ Standard Template Library

A C library of general-purpose data structures, like stacks and queues, cannot really exist because functions in C can't tell the type of their arguments. Thus we would have to define separate routines such as `push_int()` and `push_char()` for every possible data type. Further, such an approach couldn't generalize to construct stacks on user-defined data types.

Templates are C++'s mechanism to define abstract objects which can be parameterized by type. The C++ *Standard Template Library (STL)* provides implementations of all the data structures defined above and much more. Each data object must have the type of its elements fixed (i.e., templated) at compilation time, so

```
#include <stl.h>

  stack<int> S;
  stack<char> T;
```

declares two stacks with different element types.

Good references on STL include [MDS01] and *http://www.sgi.com/tech/stl/*. Brief descriptions of our featured data structures follow below –

- *Stack* — Methods include `S.push()`, `S.top()`, `S.pop()`, and `S.empty()`. Top returns but does not remove the element on top; while pop removes but does not return the element. Thus always top on pop [Seu63]. Linked implementations ensure the stack will never be full.
- *Queue* — Methods include `Q.front()`, `Q.back()`, `Q.push()`, `Q.pop()`, and `Q.empty()` and have the same idiosyncrasies as stack.
- *Dictionaries* — STL contains a wide variety of containers, including `hash_map`, a hashed associative container binding keys to data items. Methods include `H.erase()`, `H.find()`, and `H.insert()`.
- *Priority Queues* — Declared `priority_queue<int> Q;`, methods include `Q.top()`, `Q.push()`, `Q.pop()`, and `Q.empty()`.
- *Sets* — Sets are represented as sorted associative containers, declared `set<key, comparison> S;`. Set algorithms include `set_union` and `set_intersection`, as well as other standard set operators.

2.2.2 The Java `java.util` *Package*

Useful standard Java objects appear in the `java.util` package. Almost all of `java.util` is available on the judge, except for a few libraries which provide

an unhealthy degree of power to the contestant. For details, see Sun's website *http://java.sun.com/products/jdk*.

The collection of all Java classes defines an inheritance hierarchy, which means that subclasses are built on superclasses by adding methods and variables. As you move up the inheritance hierarchy, classes become more general and more abstract. The sole purpose of an *abstract* class is to provide an appropriate superclass from which other classes may inherit its interface and/or implementation. The abstract classes can only declare objects, not instantiate them. Classes from which objects can be instantiated are called *concrete* classes.

If you want to declare a general data structure object, declare it with an interface or abstract class and instantiate it using a concrete class. For example,

```
Map myMap = new HashMap();
```

In this case, `myMap` is treated as an object of `Map` class. Otherwise, you can just declare and instantiate an object with concrete class, like

```
HashMap myMap = new HashMap();
```

Here, `myMap` is only an object of `HashMap` class.

To use `java.util` include `import java.util.*;` at the beginning of your program to import the whole package, or replace the star to import a specific class, like `import java.util.HashMap;`.

Appropriate implementations of the basic data structures include —

Data Structure	Abstract class	Concrete class	Methods
Stack	No interface	`Stack`	`pop`, `push`, `empty`, `peek`
Queue	`List`	`ArrayList`, `LinkedList`	`add`, `remove`, `clear`
Dictionaries	`Map`	`HashMap`, `Hashtable`	`put`, `get`, `contains`
Priority Queue	`SortedMap`	`TreeMap`	`firstKey`, `lastKey`, `headMap`
Sets	`Set`	`HashSet`	`add`, `remove`, `contains`

2.3 Program Design Example: Going to War

In the children's card game War, a standard 52-card deck is dealt to two players (1 and 2) such that each player has 26 cards. Players do not look at their cards, but keep them in a packet face down. The object of the game is to win all the cards.

Both players play by turning their top cards face up and putting them on the table. Whoever turned the higher card takes both cards and adds them (face down) to the bottom of their packet. Cards rank as usual from high to low: *A, K, Q, J, T, 9, 8, 7, 6, 5, 4, 3, 2*. Suits are ignored. Both players then turn up their next card and repeat. The game continues until one player wins by taking all the cards.

When the face up cards are of equal rank there is a war. These cards stay on the table as both players play the next card of their pile face down and then another card face up. Whoever has the higher of the new face up cards wins the war, and adds all six cards to the bottom of his or her packet. If the new face up cards are equal as well, the war continues: each player puts another card face down and one face up. The war

goes on like this as long as the face up cards continue to be equal. As soon as they are different, the player with the higher face up card wins all the cards on the table.

If someone runs out of cards in the middle of a war, the other player automatically wins. The cards are added to the back of the winner's hand in the exact order they were dealt, specifically 1's first card, 2's first card, 1's second card, etc.

As anyone with a five year-old nephew knows, a game of War can take a long time to finish. But how long? Your job is to write a program to simulate the game and report the number of moves.

Solution starts below

How do we read such a problem description? Keep the following in mind as you design, code, test, and debug your solutions:

- *Read the Problem Carefully* — Read each line of the problem statement carefully, and reread it when the judge complains about an error. Skim the passage first, since much of the description may be background/history that does not impact the solution. Pay particular attention to the input and output descriptions, and sample input/output, but ...
- *Don't Assume* — Reading and understanding specifications is an important part of contest (and real-life) programming. Specifications often leave unspecified traps to fall in.

  Just because certain examples exhibit some nice property does not mean that all the test data will. Be on the lookout for unspecified input orders, unbounded input numbers, long line lengths, negative numbers, etc. Any input which is not explicitly forbidden should be assumed to be permitted!
- *Not So Fast, Louie* — Efficiency is often not an important issue, unless we are using exponential algorithms for problems where polynomial algorithms suffice. Don't worry too much about efficiency unless you have seen or can predict trouble. Read the specification to learn the maximum possible input size, and decide whether the most straightforward algorithm will suffice on such inputs.

  Even though a game of war seems interminable when you are playing with your nephew (and in fact can go on forever), we see no reason to worry about efficiency with this particular problem description.

2.4 Hitting the Deck

What is the best data structure to represent a deck of cards? The answer depends upon what you are going to do with them. Are you going to shuffle them? Compare their values? Search for patterns in the deck? Your intended actions define the operations of the data structure.

The primary action we need from our deck is dealing cards out from the top and adding them to the rear of our deck. Thus it is natural to represent each player's hand using the FIFO queue we defined earlier.

But there is an even more fundamental problem. How do we represent each card? Remember that cards have both suits (clubs, diamonds, hearts, and spades) and values (ace, 2–10, jack, queen, king). We have several possible choices. We can represent each card by a pair of characters or numbers specifying the suit and value. In the war problem, we might even throw away the suit entirely – but such thinking can get us in trouble. What if we had to print the winning card, or needed strong evidence that our queue implementation was working perfectly? An alternate approach might represent each card with a distinct integer, say 0 to 51, and map in back and forth between numbers and cards as needed.

The primary operation in war is comparing cards by their face value. This is tricky to do with the first character representation, because we must compare according to the historical but arbitrary ordering of face cards. Ad hoc logic might seem necessary to deal with this problem.

Instead, we will present the mapping approach as a generally useful programming technique. Whenever we can create a numerical *ranking* function and a dual *unranking* function which hold over a particular set of items $s \in S$, we can represent any item by its integer rank. The key property is that $s = unrank(rank(s))$. Thus the ranking function can be thought of as a hash function without collisions.

How can we rank and unrank playing cards? We order the card values from lowest to highest, and note that there are four distinct cards of each value. Multiplication and division are the key to mapping them from 0 to 51:

```
#define NCARDS  52      /* number of cards */
#define NSUITS  4       /* number of suits */

char values[] = "23456789TJQKA";
char suits[] = "cdhs";

int rank_card(char value, char suit)
{
        int i,j;        /* counters */

        for (i=0; i<(NCARDS/NSUITS); i++)
                if (values[i]==value)
                        for (j=0; j<NSUITS; j++)
                                if (suits[j]==suit)
                                        return( i*NSUITS + j );

        printf("Warning: bad input value=%d, suit=%d\n",value,suit);
}

char suit(int card)
{
        return( suits[card % NSUITS] );
}
```

```
char value(int card)
{
        return( values[card/NSUITS] );
}
```

Ranking and unranking functions are easy to develop for permutations, subsets, and most combinatorial objects. They are a general programming technique which can simplify operations on many different types of data.

2.5 String Input/Output

For our war program, the input consists of pairs of lines for each input deck, the first corresponding to player 1's cards, the second to player 2's cards. Here is an example of three games' worth of cards:

```
4d Ks As 4h Jh 6h Jd Qs Qh 6s 6c 2c Kc 4s Ah 3h Qd 2h 7s 9s 3c 8h Kd 7h Th Td
8d 8c 9c 7c 5d 4c Js Qc 5s Ts Jc Ad 7d Kh Tc 3s 8s 2d 2s 5h 6d Ac 5c 9h 3d 9d
6d 9d 8c 4s Kc 7c 4d Tc Kd 3s 5h 2h Ks 5c 2s Qh 8d 7d 3d Ah Js Jd 4c Jh 6c Qc
9h Qd Qs 9s Ac 8h Td Jc 7s 2d 6s As 4h Ts 6h 2c Kh Th 7h 5s 9c 5d Ad 3h 8s 3c
Ah As 4c 3s 7d Jc 5h 8s Qc Kh Td 3h 5c 9h 8c Qs 3d Ks 4d Kd 6c 6s 7h Qh 3c Jd
2h 8h 7s 2c 5d 7c 2d Tc Jh Ac 9s 9c 5s Qd 4s Js 6d Kc 2s Th 8d 9d 4h Ad 6h Ts
```

Many problems require reading non-numerical data. Text strings will be better discussed in Chapter 3, but note that you have several choices when it comes to reading text input:

- You can repeatedly get single characters from the input stream (e.g., `getchar()` in C), and process them one at a time.
- You can repeatedly get formatted tokens (e.g., `scanf()` in C) and process them as needed.
- You can read the entire line as a single string (e.g., `gets()` in C), and then parse it by accessing characters/substring in the string.
- You can use modern input/output primitives such as streams if your language supports them. Of course, you are still faced with the problem of deciding whether you want characters, strings, or something else as the basic input block.

In our war implementation, we chose the first option – namely, reading characters sequentially and processing them one by one. To make the example more illustrative, we explicitly test for end of line (`'\n'` in C):

```
main()
{
        queue decks[2];                 /* player's decks */
        char value,suit,c;              /* input characters */
        int i;                          /* deck counter */
```

```
        while (TRUE) {
            for (i=0; i<=1; i++) {
                init_queue(&decks[i]);

                while ((c = getchar()) != '\n') {
                    if (c == EOF) return;
                    if (c != ' ') {
                        value = c;
                        suit = getchar();
                        enqueue(&decks[i],rank_card(value,suit));
                    }
                }
            }

            war(&decks[0],&decks[1]);
        }
}
```

Note that we represented the two decks as an array of queues instead of as two separate queue variables. This way, we eliminate the need to duplicate all the input processing code for each deck.

2.6 Winning the War

With the proper scaffolding afforded by our data structure design, the main routine becomes fairly straightforward. Note that the order in which the conquered cards get returned to the winning deck also can be modeled as a queue, so we get to use our abstract type once again:

```
war(queue *a, queue *b)
{
        int steps=0;                    /* step counter */
        int x,y;                        /* top cards */
        queue c;                        /* cards involved in the war */
        bool inwar;                     /* are we involved in a war? */

        inwar = FALSE;
        init_queue(&c);

        while ((!empty(a)) && (!empty(b) && (steps < MAXSTEPS))) {
                steps = steps + 1;
                x = dequeue(a);
                y = dequeue(b);
                enqueue(&c,x);
```

```
                enqueue(&c,y);
                if (inwar) {
                        inwar = FALSE;
                } else {
                        if (value(x) > value(y))
                                clear_queue(&c,a);
                        else if  (value(x) < value(y))
                                clear_queue(&c,b);
                        else if (value(y) == value(x))
                                inwar = TRUE;
                }
        }

        if (!empty(a) && empty(b))
                printf("a wins in %d steps \n",steps);
        else if (empty(a) && !empty(b))
                printf("b wins in %d steps \n",steps);
        else if (!empty(a) && !empty(b))
                printf("game tied after %d steps, |a|=%d |b|=%d \n",
                        steps,a->count,b->count);
        else
                printf("a and b tie in %d steps \n",steps);
}

clear_queue(queue *a, queue *b)
{
        while (!empty(a)) enqueue(b,dequeue(a));
}
```

2.7 Testing and Debugging

Debugging can be particularly frustrating with the robot judge, since you never get to see the test case on which your program failed. Thus you can't fake your way around a problem – you have to do it right.

This makes it very important to test your program systematically before submission. Catching stupid errors yourself will save you time in the long run, particularly in contest situations where incorrect submissions are penalized. Several ideas go into designing good test files:

- *Test the Given Input* — Most problem specifications include sample input and output. Often (but not always) they match each other. Getting the sample input right is a necessary but not sufficient condition for correctness.
- *Test Incorrect Input* — If the problem specification tells you that your program must take action on illegal inputs, be sure to test for such problematic instances.

- *Test Boundary Conditions* — Many bugs in programs are due to "off-by-one" errors. Explicitly test your code for such conditions as empty input, one item, two items, and values which are zero.
- *Test Instances Where You Know the Correct Answer* — A critical part of developing a good test case is being sure you know what the right answer is. Your test cases should focus on simple enough instances that you can solve them by hand. It is easy to be fooled into accepting plausible-looking output without completely analyzing the desired behavior.
- *Test Big Examples Where You Don't Know the Correct Answer* — Usually only small examples are solvable by hand. This makes it difficult to validate your program on larger inputs. Try a few easily constructed large instances, such as random data or the numbers 1 to n inclusive, just to make certain the program doesn't crash or do anything stupid.

Testing is the art of revealing bugs. Debugging is the art of exterminating them. We designed this programming problem and coded it up ourselves for the purpose of this example. Yet getting it working without bugs took us significantly longer than expected. This is no surprise, for all programmers are inherent optimists. But how can you avoid falling into such traps?

- *Get To Know Your Debugger* — Any reasonable programming environment comes with a source-level debugger, which lets you stop execution at given positions or logical conditions, look at the contents of a variable, and change its value to see what happens. Source-level debuggers are well worth their weight in print statements; learn to use them. The sooner you start, the more time and frustration you will save.
- *Display Your Data Structures* — At one point in debugging our War program, we had an off-by-one error in our priority queue. This could only be revealed by displaying the contents of the priority queue to see what was missing. Write special-purpose display routines for all non-trivial data structures, as debuggers often have a hard time making sense out of them.
- *Test Invariants Rigorously* — The card ranking and unranking functions are a potential source of error whenever they are not inverses of each other. An *invariant* is a property of the program which is true regardless of the input. A simple invariant test like

```
for (i=0; i<NCARDS; i++)
   if (i != rank_card(value(i), suit(i)))
        printf("Error: rank_card(%c,%c)=%d not %d\n", value(i),
                suit(i), rank_card(value(i), suit(i)), i);
```

  completely tests the correctness of the ranking and unranking functions.
- *Inspect Your Code* — Perhaps the single most powerful way to debug your program is to read it carefully. Bugs tend to infest code that is too ugly to read or too clever to understand.

- *Make Your Print Statements Mean Something* — Adding print statements to peer into the internals of your malfunctioning program is a necessary evil which can be minimized by effective use of a source-level debugger. But if you are going to add debugging print statements, make them as useful as possible. Be sure to display with the names of the variable and its position in the program as well as its value. It is easy to get lost in an enormous volume of debugging output, but the relevant lines can be quickly found by searching or greping through the output file.

  Comment out your debugging statements when you have fixed that part of the program, but don't be so fast to delete them. If you are like us, you will probably need them again.

- *Make Your Arrays a Little Larger Than Necessary* — Off-by-one errors are a particularly subtle and tricky class of bugs. Clear thinking and discipline are the right way avoid them. But thinking is hard and memory is cheap. We find it a useful crutch to define arrays one or two elements larger than *should* be necessary so as to minimize the consequences of any off-by-one error.

- *Make Sure Your Bugs are Really Bugs* — We spent some time tracking down an infinite loop in the War program, before discovering that random initial card orderings have an astonishingly high probability of creating a cycling condition where players forever alternate taking cards from each other. The culprit was the deterministic card order specified to collect the spoils of war, i.e., always interleaving player 1 and player 2's cards. In fact, there was no bug in the program at all!

  Please learn from this example. Always shuffle the pickup cards randomly to avoid having to play War with your nephew forever.

2.8 Problems

2.8.1 Jolly Jumpers

PC/UVa IDs: 110201/10038, **Popularity:** A, **Success rate:** average **Level:** 1

A sequence of $n > 0$ integers is called a *jolly jumper* if the absolute values of the differences between successive elements take on all possible values 1 through $n - 1$. For instance,

```
1 4 2 3
```

is a jolly jumper, because the absolute differences are 3, 2, and 1, respectively. The definition implies that any sequence of a single integer is a jolly jumper. Write a program to determine whether each of a number of sequences is a jolly jumper.

Input

Each line of input contains an integer $n < 3,000$ followed by n integers representing the sequence.

Output

For each line of input generate a line of output saying "`Jolly`" or "`Not jolly`".

Sample Input

```
4 1 4 2 3
5 1 4 2 -1 6
```

Sample Output

```
Jolly
Not jolly
```

2.8.2 Poker Hands

PC/UVa IDs: 110202/10315, **Popularity:** C, **Success rate:** average **Level:** 2

A poker deck contains 52 cards. Each card has a suit of either clubs, diamonds, hearts, or spades (denoted `C`, `D`, `H`, `S` in the input data). Each card also has a value of either 2 through 10, jack, queen, king, or ace (denoted `2`, `3`, `4`, `5`, `6`, `7`, `8`, `9`, `T`, `J`, `Q`, `K`, `A`). For scoring purposes card values are ordered as above, with 2 having the lowest and ace the highest value. The suit has no impact on value.

A poker hand consists of five cards dealt from the deck. Poker hands are ranked by the following partial order from lowest to highest.

High Card. Hands which do not fit any higher category are ranked by the value of their highest card. If the highest cards have the same value, the hands are ranked by the next highest, and so on.

Pair. Two of the five cards in the hand have the same value. Hands which both contain a pair are ranked by the value of the cards forming the pair. If these values are the same, the hands are ranked by the values of the cards not forming the pair, in decreasing order.

Two Pairs. The hand contains two different pairs. Hands which both contain two pairs are ranked by the value of their highest pair. Hands with the same highest pair are ranked by the value of their other pair. If these values are the same the hands are ranked by the value of the remaining card.

Three of a Kind. Three of the cards in the hand have the same value. Hands which both contain three of a kind are ranked by the value of the three cards.

Straight. Hand contains five cards with consecutive values. Hands which both contain a straight are ranked by their highest card.

Flush. Hand contains five cards of the same suit. Hands which are both flushes are ranked using the rules for High Card.

Full House. Three cards of the same value, with the remaining two cards forming a pair. Ranked by the value of the three cards.

Four of a Kind. Four cards with the same value. Ranked by the value of the four cards.

Straight Flush. Five cards of the same suit with consecutive values. Ranked by the highest card in the hand.

Your job is to compare several pairs of poker hands and to indicate which, if either, has a higher rank.

Input

The input file contains several lines, each containing the designation of ten cards: the first five cards are the hand for the player named "*Black*" and the next five cards are the hand for the player named "*White*".

Output

For each line of input, print a line containing one of the following:

```
Black wins.
White wins.
Tie.
```

Sample Input

```
2H 3D 5S 9C KD 2C 3H 4S 8C AH
2H 4S 4C 2D 4H 2S 8S AS QS 3S
2H 3D 5S 9C KD 2C 3H 4S 8C KH
2H 3D 5S 9C KD 2D 3H 5C 9S KH
```

Sample Output

```
White wins.
Black wins.
Black wins.
Tie.
```

2.8.3 Hartals

PC/UVa IDs: 110203/10050, **Popularity:** B, **Success rate:** high **Level:** 2

Political parties in Bangladesh show their muscle by calling for regular *hartals* (strikes), which cause considerable economic damage. For our purposes, each party may be characterized by a positive integer h called the *hartal parameter* that denotes the average number of days between two successive strikes called by the given party.

Consider three political parties. Assume $h_1 = 3$, $h_2 = 4$, and $h_3 = 8$, where h_i is the hartal parameter for party i. We can simulate the behavior of these three parties for $N = 14$ days. We always start the simulation on a Sunday. There are no hartals on either Fridays or Saturdays.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Days	Su	Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa
Party 1			x			x			x			x		
Party 2				x				x				x		
Party 3								x						
Hartals			1	2				3	4			5		

There will be exactly five hartals (on days 3, 4, 8, 9, and 12) over the 14 days. There is no hartal on day 6 since it falls on Friday. Hence we lose five working days in two weeks.

Given the hartal parameters for several political parties and the value of N, determine the number of working days lost in those N days.

Input

The first line of the input consists of a single integer T giving the number of test cases to follow. The first line of each test case contains an integer N ($7 \leq N \leq 3,650$), giving the number of days over which the simulation must be run. The next line contains another integer P ($1 \leq P \leq 100$) representing the number of political parties. The ith of the next P lines contains a positive integer h_i (which will never be a multiple of 7) giving the *hartal parameter* for party i ($1 \leq i \leq P$).

Output

For each test case, output the number of working days lost on a separate line.

Sample Input

```
2
14
3
3
4
```

```
8
100
4
12
15
25
40
```

Sample Output

```
5
15
```

2.8.4 Crypt Kicker

PC/UVa IDs: 110204/843, **Popularity:** B, **Success rate:** low **Level:** 2

A common but insecure method of encrypting text is to permute the letters of the alphabet. In other words, each letter of the alphabet is consistently replaced in the text by some other letter. To ensure that the encryption is reversible, no two letters are replaced by the same letter.

Your task is to decrypt several encoded lines of text, assuming that each line uses a different set of replacements, and that all words in the decrypted text are from a dictionary of known words.

Input

The input consists of a line containing an integer n, followed by n lowercase words, one per line, in alphabetical order. These n words compose the dictionary of words which may appear in the decrypted text. Following the dictionary are several lines of input. Each line is encrypted as described above.

There are no more than 1,000 words in the dictionary. No word exceeds 16 letters. The encrypted lines contain only lower case letters and spaces and do not exceed 80 characters in length.

Output

Decrypt each line and print it to standard output. If there are multiple solutions, any one will do. If there is no solution, replace every letter of the alphabet by an asterisk.

Sample Input

```
6
and
dick
jane
puff
spot
yertle
bjvg xsb hxsn xsb qymm xsb rqat xsb pnetfn
xxxx yyy zzzz www yyyy aaa bbbb ccc dddddd
```

Sample Output

```
dick and jane and puff and spot and yertle
**** *** **** *** **** *** **** *** ******
```

2.8.5 Stack 'em Up

PC/UVa IDs: 110205/10205, **Popularity:** B, **Success rate:** average **Level:** 1

The Big City has many casinos. In one of them, the dealer cheats. She has perfected several shuffles; each shuffle rearranges the cards in exactly the same way whenever it is used. A simple example is the "bottom card" shuffle, which removes the bottom card and places it at the top. By using various combinations of these known shuffles, the crooked dealer can arrange to stack the cards in just about any particular order.

You have been retained by the security manager to track this dealer. You are given a list of all the shuffles performed by the dealer, along with visual cues that allow you to determine which shuffle she uses at any particular time. Your job is to predict the order of the cards after a sequence of shuffles.

A standard playing card deck contains 52 cards, with 13 values in each of four suits. The values are named *2, 3, 4, 5, 6, 7, 8, 9, 10, Jack, Queen, King, Ace*. The suits are named *Clubs, Diamonds, Hearts, Spades*. A particular card in the deck can be uniquely identified by its value and suit, typically denoted $< value >$ of $< suit >$. For example, "9 of Hearts" or "King of Spades." Traditionally a new deck is ordered first alphabetically by suit, then by value in the order given above.

Input

The input begins with a single positive integer on a line by itself indicating the number of test cases, followed by a blank line. There is also a blank line between two consecutive inputs.

Each case consists of an integer $n \leq 100$, the number of shuffles that the dealer knows. Then follow n sets of 52 integers, each comprising all the integers from 1 to 52 in some order. Within each set of 52 integers, i in position j means that the shuffle moves the ith card in the deck to position j.

Several lines follow, each containing an integer k between 1 and n. These indicate that you have observed the dealer applying the kth shuffle given in the input.

Output

For each test case, assume the dealer starts with a new deck ordered as described above. After all the shuffles had been performed, give the names of the cards in the deck, in the new order. The output of two consecutive cases will be separated by a blank line.

Sample Input

```
1

2
2 1 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26
27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 52 51
52 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26
```

```
27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 1
1
2
```

Sample Output

```
King of Spades
2 of Clubs
4 of Clubs
5 of Clubs
6 of Clubs
7 of Clubs
8 of Clubs
9 of Clubs
10 of Clubs
Jack of Clubs
Queen of Clubs
King of Clubs
Ace of Clubs
2 of Diamonds
3 of Diamonds
4 of Diamonds
5 of Diamonds
6 of Diamonds
7 of Diamonds
8 of Diamonds
9 of Diamonds
10 of Diamonds
Jack of Diamonds
Queen of Diamonds
King of Diamonds
Ace of Diamonds
2 of Hearts
3 of Hearts
4 of Hearts
5 of Hearts
6 of Hearts
7 of Hearts
8 of Hearts
9 of Hearts
10 of Hearts
Jack of Hearts
Queen of Hearts
King of Hearts
Ace of Hearts
2 of Spades
3 of Spades
4 of Spades
5 of Spades
6 of Spades
7 of Spades
8 of Spades
9 of Spades
10 of Spades
Jack of Spades
Queen of Spades
Ace of Spades
3 of Clubs
```

2.8.6 Erdös Numbers

PC/UVa IDs: 110206/10044, **Popularity:** B, **Success rate:** low **Level:** 2

The Hungarian Paul Erdös (1913–1996) was one of the most famous mathematicians of the 20th century. Every mathematician having the honor of being a co-author of Erdös is well respected.

Unfortunately, not everybody got the chance to write a paper with Erdös, so the best they could do was publish a paper with somebody who had published a scientific paper with Erdös. This gave rise to the so-called *Erdös numbers*. An author who has jointly published with Erdös had Erdös number 1. An author who had not published with Erdös but with somebody with Erdös number 1 obtained Erdös number 2, and so on.

Your task is to write a program which computes Erdös numbers for a given set of papers and scientists.

Input

The first line of the input contains the number of scenarios. Each scenario consists of a paper database and a list of names. It begins with the line P N, where P and N are natural numbers. Following this line is the paper database, with P lines each containing the description of one paper specified in the following way:

```
Smith, M.N., Martin, G., Erdos, P.: Newtonian forms of prime factors
```

Note that umlauts, like "ö," are simply written as "o". After the P papers follow N lines with names. Such a name line has the following format:

```
Martin, G.
```

Output

For every scenario you are to print a line containing a string "`Scenario` i" (where i is the number of the scenario) and the author names together with their Erdös number of all authors in the list of names. The authors should appear in the same order as they appear in the list of names. The Erdös number is based on the papers in the paper database of this scenario. Authors which do not have any relation to Erdös via the papers in the database have Erdös number "`infinity`."

Sample Input

```
1
4 3
Smith, M.N., Martin, G., Erdos, P.: Newtonian forms of prime factors
Erdos, P., Reisig, W.: Stuttering in petri nets
Smith, M.N., Chen, X.: First order derivates in structured programming
Jablonski, T., Hsueh, Z.: Selfstabilizing data structures
```

```
Smith, M.N.
Hsueh, Z.
Chen, X.
```

Sample Output

```
Scenario 1
Smith, M.N. 1
Hsueh, Z. infinity
Chen, X. 2
```

2.8.7 Contest Scoreboard

PC/UVa IDs: 110207/10258, **Popularity:** B, **Success rate:** average **Level:** 1

Want to compete in the ACM ICPC? Then you had better know how to keep score! Contestants are ranked first by the number of problems solved (the more the better), then by decreasing amounts of penalty time. If two or more contestants are tied in both problems solved and penalty time, they are displayed in order of increasing team numbers.

A problem is considered solved by a contestant if any of the submissions for that problem was judged correct. Penalty time is computed as the number of minutes it took until the first correct submission for a problem was received, plus 20 minutes for each incorrect submission prior to the correct solution. Unsolved problems incur no time penalties.

Input

The input begins with a single positive integer on a line by itself indicating the number of cases, each described as below. This line is followed by a blank line. There is also a blank line between two consecutive inputs.

The input consists of a snapshot of the judging queue, containing entries from some or all of contestants 1 through 100 solving problems 1 through 9. Each line of input consists of three numbers and a letter in the format *contestant problem time L*, where *L* can be `C`, `I`, `R`, `U`, or `E`. These stand for `C`orrect, `I`ncorrect, clarification `R`equest, `U`njudged, and `E`rroneous submission. The last three cases do not affect scoring.

The lines of input appear in the order in which the submissions were received.

Output

The output for each test case will consist of a scoreboard, sorted by the criteria described above. Each line of output will contain a contestant number, the number of problems solved by the contestant and the total time penalty accumulated by the contestant. Since not all contestants are actually participating, only display those contestants who have made a submission.

The output of two consecutive cases will be separated by a blank line.

Sample Input

```
1

1 2 10 I
3 1 11 C
1 2 19 R
1 2 21 C
1 1 25 C
```

Sample Output

```
1 2 66
3 1 11
```

2.8.8 Yahtzee

PC/UVa IDs: 110208/10149, **Popularity:** C, **Success rate:** average **Level:** 3

The game of Yahtzee involves five dice, which are thrown in 13 rounds. A score card contains 13 categories. Each round may be scored in a category of the player's choosing, but each category may be scored only once in the game. The 13 categories are scored as follows:

- **ones** - sum of all ones thrown
- **twos** - sum of all twos thrown
- **threes** - sum of all threes thrown
- **fours** - sum of all fours thrown
- **fives** - sum of all fives thrown
- **sixes** - sum of all sixes thrown
- **chance** - sum of all dice
- **three of a kind** - sum of all dice, provided at least three have same value
- **four of a kind** - sum of all dice, provided at least four have same value
- **five of a kind** - 50 points, provided all five dice have same value
- **short straight** - 25 points, provided four of the dice form a sequence (that is, 1,2,3,4 or 2,3,4,5 or 3,4,5,6)
- **long straight** - 35 points, provided all dice form a sequence (1,2,3,4,5 or 2,3,4,5,6)
- **full house** - 40 points, provided three of the dice are equal and the other two dice are also equal. (for example, 2,2,5,5,5)

Each of the last six categories may be scored as 0 if the criteria are not met.

The score for the game is the sum of all 13 categories plus a bonus of 35 points if the sum of the first six categories is 63 or greater.

Your job is to compute the best possible score for a sequence of rounds.

Input

Each line of input contains five integers between 1 and 6, indicating the values of the five dice thrown in each round. There are 13 such lines for each game, and there may be any number of games in the input data.

Output

Your output should consist of a single line for each game containing 15 numbers: the score in each category (in the order given), the bonus score (0 or 35), and the total score. If there is more than categorization that yields the same total score, any one will do.

Sample Input

```
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 1 1 1 1
6 6 6 6 6
6 6 6 1 1
1 1 1 2 2
1 1 1 2 3
1 2 3 4 5
1 2 3 4 6
6 1 2 6 6
1 4 5 5 5
5 5 5 5 6
4 4 4 5 6
3 1 3 6 3
2 2 2 4 6
```

Sample Output

```
1 2 3 4 5 0 15 0 0 0 25 35 0 0 90
3 6 9 12 15 30 21 20 26 50 25 35 40 35 327
```

2.9 Hints

2.2 Can we reduce the value of a poker hand to a single numerical value to make the comparison easier?

2.3 Do we need to build the actual table in order to compute the number of hartals?

2.4 Does it pay to partition the words into equivalence classes based on repeated letters and length?

2.7 What is the easiest way to sort on the multiple criteria?

2.8 Do we need to try all possible mappings of rounds to categories, or can we make certain assignments in a more straightforward way?

2.10 Notes

2.1 A jolly number is a special case of a *graceful graph* labeling. A graph is called graceful if there exists a way to label the n vertices with integers such that the absolute value of the difference between the endpoints of all m of the edges yields a distinct value between 1 and m. A jolly jumper represents a graceful labeling of an n-vertex path. The famous *graceful tree conjecture* asks whether every tree has a graceful labeling. Graceful graphs make a great topic for undergraduate student research. See Gallian's dynamic survey [Gal01] for a list of accessible open problems.

2.5 The mathematics of card shuffling is a fascinating topic. A *perfect shuffle* splits the deck in half into piles A and B, and then interleaves the cards: top of A, top of B, top of A, ... Amazingly, eight perfect shuffles take a deck back to its original state! This can be shown using either modular arithmetic or the theory of cycles in permutations. See [DGK83, Mor98] for more on perfect shuffles.

2.6 The first author of this book has an Erdös number of 2, giving the second a number of ≤ 3. Erdös was famous for posing beautiful, easy-to-understand but hard-to-solve problems in combinatorics, graph theory, and number theory. Read one of the popular biographies of his life [Hof99, Sch00] for more about this fascinating man.